Chapter 1

Tree Traversal

Trees can be "traversed" in a few different ways. When we say traversing a tree, we mean that we are visiting and extracting data from each node in a tree. We classify these *traversals* based on the order in which the nodes are visited.

This chapter will discuss *breadth-first search*, also known as *BFS*, which traverses in a *level-order*. Each level is traversed in sequential order.

Alternatively, there is *depth-first search* (*DFS*), which travels as far down a branch as possible before backtracking and checking other branches. Within DFS there are 3 common traversal methodologies: *in-order*, *pre-order* and *post-order*.

While the demonstrations for these traversals will be for trees, they work the same for any given graph, under one condition: using DFS or BFS on a *graph* requires you do have a preselected node a your *start node*.

1.1 BFS

Let's look at running BFS on a tree. Starting out with this tree:

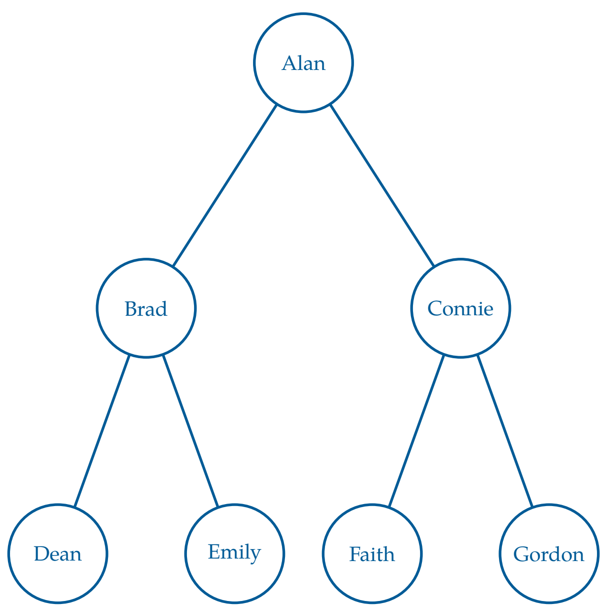

Figure 1.1: Our original tree containing a list of names to be traversed.

To perform BFS, we use a *queue*, which follows a first-in, first-out (*FIFO*) order. We begin by placing the root node into the queue. At each step, we remove the front node from the queue, visit it, and then add all of its children to the *back* of the queue.

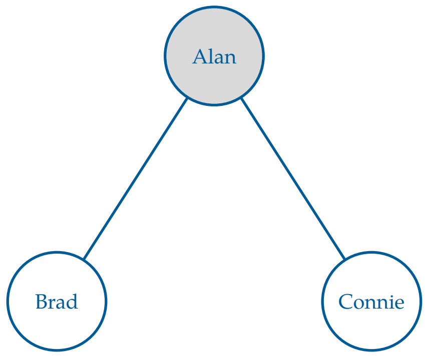

Figure 1.2: Step 1: Visit Alan, then add its children (Brad, Connie) to the queue.

Here, we visit the root node first, which is Alan. After visiting Alan, we add its children, Brad and Connie, to the queue. Since there are no other nodes to visit at this level, we move on to the next level of the tree.

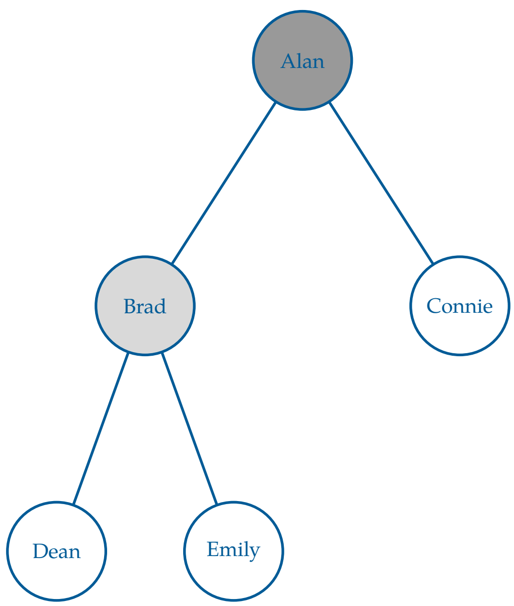

Figure 1.3: Step 2: Visit Brad, then add its children (Dean, Emily) to the queue.

We continue this process, visiting Connie next and adding its children, Faith and Gordon, to the queue. The queue now contains Connie, Dean, and Emily. Here is a step trace of the BFS traversal:

Step	Queue	Output (visited)
Start	[Alan]	[]
1	[Brad, Connie]	[Alan]
2	[Connie, Dean, Emily]	[Alan, Brad]
3	[Dean, Emily, Faith, Gordon]	[Alan, Brad, Connie]
4	[Emily, Faith, Gordon]	[Alan, Brad, Connie, Dean]

Continuing this process until the queue is empty gives the final traversal:

Alan, Brad, Connie, Dean, Emily, Faith, Gordon

This ordering reflects a level-by-level traversal of the tree, which is why BFS on a tree is also called level-order traversal.

FIXME exercise - run DFS on a tree

Our queue steps can be reflected as pseudocode as follows:

Algorithm 1 Breadth-First Search (BFS)

```
procedure BFS(root)
    Q ← empty queue
    Enqueue(Q, root)
    while Q is not empty do
        node ← Dequeue(Q)
        Visit(node)
        for each child of node do
            Enqueue(Q, child)
        end for
    end while
end procedure
```

This procedure is often used in game applications to find paths, such as maze solvers, chess algorithms, and tic-tac-toe solutions. It is also used in social network analysis to find the shortest path between two users, and in web crawling to explore the structure of websites.

1.2 DFS

1.2.1 Pre-order

1.2.2 Post-order

1.2.3 In-order

This is a draft chapter from the Kontinua Project. Please see our website (https://kontinua.org/) for more details.

Appendix A

Answers to Exercises

Index